

STUDENT EXECUTION GUIDE - WHERE TO TYPE AND WHERE TO CREATE FILES

The original lab manual content is retained. The added pages remove ambiguity about the exact terminal, directory, and file location to use while following the original steps.

Terminal rule used throughout the experiment

- For Linux commands, open the Ubuntu Terminal. When the manual says 'Inside the VM', type the command in the Ubuntu terminal running inside VMware Workstation.
- For Docker commands, type them in the terminal where Docker is installed. Keep that choice fixed and record it in the experiment documentation.
- When a command uses a relative path such as `scripts/run_cpu.sh`, first move to the project root:
`cd ~/vm-vs-container-performance`.
- Keep all project files under `~/vm-vs-container-performance` so the GitHub repository has one consistent structure.

Use the following project root before creating or running project files:

```
cd ~/vm-vs-container-performance
pwd
```

The output should look similar to:

```
/home/<your-ubuntu-user>/vm-vs-container-performance
```

- The username will be different for each student.
- Keep the folder name `vm-vs-container-performance` unchanged throughout the experiment.
- If `pwd` shows another folder, run the `cd` command again before using a relative path.

PERFORMANCE ANALYSIS OF VIRTUAL MACHINES AND CONTAINERS

Complete Practical / Lab Manual

Project Objective

To experimentally compare the performance, resource utilization, application performance, and scalability of Virtual Machines (VMs) and Containers under identical workloads.

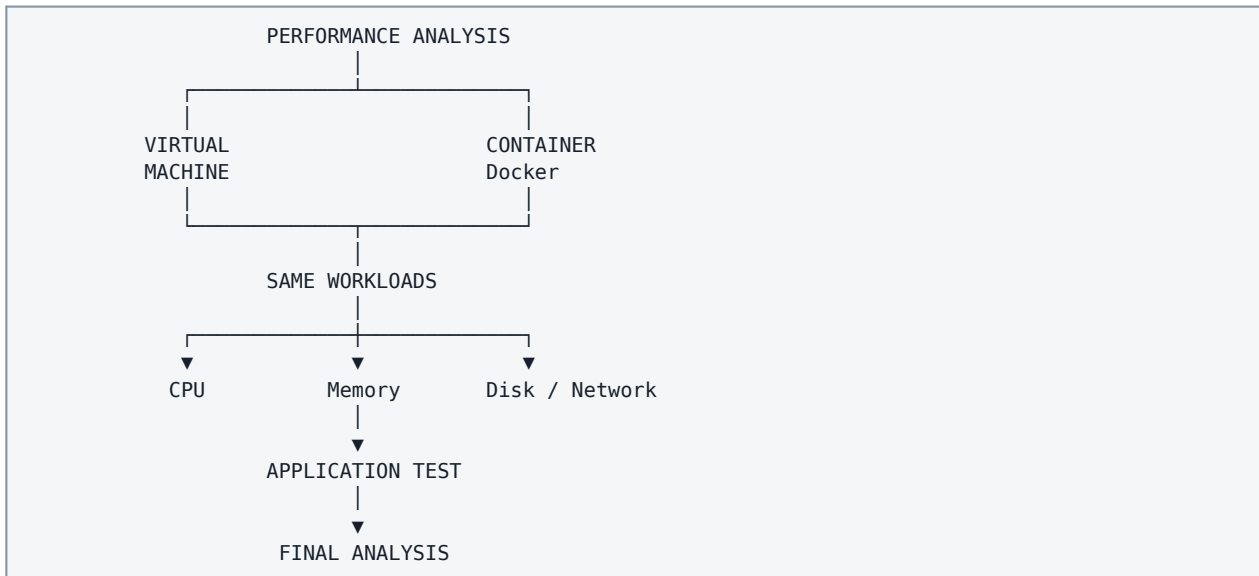
Experiments Covered

- Experimental environment preparation and configuration
- Baseline measurement
- CPU performance using Sysbench
- Memory performance using Sysbench
- Disk I/O performance using fio
- Network performance using iperf3
- FastAPI application performance
- Application and environment startup time
- Scalability with increasing workloads
- Automated benchmark execution
- CSV-based result processing and statistical analysis
- Graphs, comparison tables, and GitHub publication

This manual is designed to be followed sequentially. Complete and verify each experiment before moving to the next one.

1. Project Overview

The project compares two deployment environments running the same workloads: a Virtual Machine environment and a Docker Container environment.



- The same workload is executed in both environments so that the comparison is meaningful.
- Each experiment is repeated multiple times to reduce the effect of temporary system fluctuations.
- Raw data, processed results, graphs, scripts, and documentation are maintained so that the experiment can be reproduced.

2. Experimental Environment

EXPERIMENTAL ENVIRONMENT	
Host OS	→ Windows
Hypervisor	→ VMware Workstation
Guest OS	→ Ubuntu
Container	→ Docker
Programming	→ Python
CPU Benchmark	→ Sysbench
Disk Benchmark	→ fio
Network	→ iperf3
API	→ FastAPI
Analysis	→ Pandas / Matplotlib

Install benchmark tools in Ubuntu:

```
sudo apt update

sudo apt install -y \
    sysbench \
    fio \
    iperf3 \
    htop \
    iotop \
    sysstat \
    python3 \
    python3-pip \
    git
```

Verify the installations:

```
sysbench --version
fio --version
iperf3 --version
python3 --version
git --version
```

Check the system:

```
lscpu
free -h
lsblk
df -h
uname -a
```

Create the project directory:

```
mkdir -p ~/vm-vs-container-performance
cd ~/vm-vs-container-performance
```

- These commands install the tools that will be used throughout the experiment.
- The system commands record CPU, memory, storage, and kernel information needed for the methodology.
- Keep the environment consistent throughout the experiments.

3. Record Hardware and Software Configuration

```
mkdir -p docs results/raw results/processed results/figures scripts workloads
```

```
lscpu > docs/cpu-info.txt
free -h > docs/memory-info.txt
lsblk > docs/storage-info.txt
uname -a > docs/kernel-info.txt
```

```
docker --version
docker info
```

- These commands create a permanent record of the environment used for benchmarking.
- Benchmark results are meaningful only when the underlying hardware and software configuration is known.
- The recorded information can be included in the final report and GitHub README.

4. Create the VM Environment

Create an Ubuntu VM using VMware Workstation. Use fixed resource allocation throughout the experiments.

VM CONFIGURATION	
CPU	→ Fixed vCPU allocation
Memory	→ Fixed RAM allocation
Disk	→ Fixed virtual disk
OS	→ Ubuntu
Network	→ Fixed network configuration

RECOMMENDED CONTROLLED VM CONFIGURATION

The original manual requires fixed CPU, memory, disk and network allocation. The following is a practical example configuration for a controlled lab comparison. These are recommended example values, not measured results; record the actual values used on the system.

Recommended VM values

vCPU: 4 virtual CPUs

Memory: 8 GB RAM

Virtual disk: 60 GB

Guest OS: Ubuntu 24.04 LTS

Network: choose NAT or Bridged and keep the choice unchanged

In VMware Workstation, open the VM settings before the benchmark campaign:

```
VMware Workstation
-> Open the Ubuntu VM
-> Edit virtual machine settings
-> Processors      = 4
-> Memory          = 8 GB
-> Hard Disk       = 60 GB
-> Network Adapter = NAT or Bridged
-> Apply / Save
-> Power on the VM
```

- After Ubuntu starts, open the Ubuntu Terminal and run `nproc` and `free -h` to confirm the assigned CPU and memory.
- Run `lsblk` and `df -h` to confirm the virtual disk is visible and record the result.
- Write the final VM settings into the project documentation before starting the benchmark runs.

```
cd ~/vm-vs-container-performance
mkdir -p docs
nano docs/vm-configuration.txt
```

Type the actual VM values in this file, save with `Ctrl+O`, press `Enter`, then exit with `Ctrl+X`.

Inside the VM:

```
sudo apt update
sudo apt upgrade -y
```

```
sudo apt install -y \
    sysbench \
    fio \
    iperf3 \
    htop \
    sysstat \
    python3 \
    python3-pip \
    git
```

```
sysbench --version
fio --version
iperf3 --version
python3 --version

nproc
free -h
```

- The VM represents the traditional virtualization environment.
- Keep the VM CPU and memory allocation fixed; changing resources between tests would affect the comparison.
- Verify all benchmark tools before starting the actual experiments.

5. Create the Docker Environment

Install Docker if it is not already installed:

```
sudo apt update

sudo apt install -y docker.io

sudo systemctl enable --now docker
```

Check Docker:

```
docker --version
```

Test Docker:

```
sudo docker run --rm hello-world
```

Allow the current user to run Docker without sudo:

```
sudo usermod -aG docker $USER
```

After logging out and back in, verify:

```
docker run --rm hello-world
```

- Docker provides the container environment used for comparison.
- The hello-world test confirms that Docker is working before benchmark experiments begin.
- The container should execute the same workloads and benchmark tools used in the VM.

RECOMMENDED CONTROLLED CONTAINER CONFIGURATION

Containers can use host resources by default. To make the resource comparison controlled, apply explicit CPU and memory limits to the benchmark container and document the storage and network configuration.

Recommended container values

CPU limit: 4 CPUs

Memory limit: 8 GB

Base benchmark image: ubuntu:24.04

Storage: dedicated benchmark directory mounted into the container

Network: choose and document the mode used for the test

Run the benchmark container with these limits when performing the controlled comparison:

```
docker run --rm \
  --cpus=4 \
  --memory=8g \
  vm-container-benchmark
```

For a CPU benchmark, place the resource options before the image name:

```
docker run --rm \
  --cpus=4 \
  --memory=8g \
  vm-container-benchmark \
  sysbench cpu \
  --cpu-max-prime=20000 \
  --threads=4 \
  --time=30 \
  run
```

- Use the same CPU and memory limits throughout the controlled container experiments.
- Document the container limits because they become part of the experimental configuration.
- For disk tests, document the host directory or Docker volume used because storage placement can change the measured result.

6. Create the Benchmark Docker Image

```
mkdir -p docker
cd docker
```

Create a file named Dockerfile:

```
FROM ubuntu:24.04

RUN apt-get update && \
    apt-get install -y \
    sysbench \
    fio \
    iperf3 \
    python3 \
    python3-pip \
    procps \
    sysstat && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /benchmark
```

Build the image:

```
docker build -t vm-container-benchmark .
```

Check the image:

```
docker images
```

Run it:

```
docker run --rm -it vm-container-benchmark
```

Inside the container:

```
sysbench --version
fio --version
python3 --version
```

- The Dockerfile creates a standardized benchmark environment.
- The same image can be reused for every container experiment.
- Containerizing the benchmark environment improves reproducibility.

STEP 6 - EXACT DOCKERFILE LOCATION

When the original manual says 'Create a file named Dockerfile', create it inside the project's docker directory.

```
cd ~/vm-vs-container-performance
mkdir -p docker
cd docker
pwd
```

The final location should be similar to:

```
/home/<your-ubuntu-user>/vm-vs-container-performance/docker
```

Create the Dockerfile in that directory:

```
nano Dockerfile
```

Paste the Dockerfile shown in the original manual. Save with Ctrl+O, press Enter, and exit with Ctrl+X.

Return to the project root before building the image:

```
cd ~/vm-vs-container-performance
docker build -t vm-container-benchmark -f docker/Dockerfile .
```

- The `-f docker/Dockerfile` option tells Docker exactly which Dockerfile to use.
- The final dot uses the project root as the Docker build context.
- Keep Dockerfile inside `docker/` so the GitHub structure stays consistent with the manual.

7. Establish the Baseline

Before comparing VM and container performance, establish a baseline using the same CPU workload.

```
sysbench cpu \
  --cpu-max-prime=20000 \
  --threads=4 \
  --time=30 \
  run
```

Save the result:

```
mkdir -p results/raw/baseline

sysbench cpu \
  --cpu-max-prime=20000 \
  --threads=4 \
  --time=30 \
  run > results/raw/baseline/cpu.txt
```

- The baseline provides a reference point for interpreting VM and container results.
- Use the same benchmark parameters in later experiments.
- Save the complete output instead of recording only one number.

8. Experiment 1 — CPU Performance

CPU EXPERIMENT	
Tool	→ Sysbench
Workload	→ Prime calculation
Threads	→ 1 / 2 / 4 / 8
Duration	→ 30 seconds
Metrics	→ Events/sec + execution time

Prepare VM result directory:

```
mkdir -p ~/vm-vs-container-performance/results/raw/cpu/vm
```

Run one VM test:

```
sysbench cpu \
  --cpu-max-prime=20000 \
  --threads=4 \
  --time=30 \
  run
```

Save the result:

```
sysbench cpu \
  --cpu-max-prime=20000 \
  --threads=4 \
  --time=30 \
  run > ~/vm-vs-container-performance/results/raw/cpu/vm/run1.txt
```

Run 10 repetitions:

```

for i in {1..10}
do
    sysbench cpu \
        --cpu-max-prime=20000 \
        --threads=4 \
        --time=30 \
        run > ~/vm-vs-container-performance/results/raw/cpu/vm/run$i.txt
done

```

Prepare container result directory:

```
mkdir -p ~/vm-vs-container-performance/results/raw/cpu/container
```

Run the container test:

```

docker run --rm \
    vm-container-benchmark \
    sysbench cpu \
        --cpu-max-prime=20000 \
        --threads=4 \
        --time=30

```

Run the container test 10 times:

```

for i in {1..10}
do
    docker run --rm \
        vm-container-benchmark \
        sysbench cpu \
            --cpu-max-prime=20000 \
            --threads=4 \
            --time=30 \
            > ~/vm-vs-container-performance/results/raw/cpu/container/run$i.txt
done

```

Test CPU scalability:

```

for threads in 1 2 4 8
do
    sysbench cpu \
        --cpu-max-prime=20000 \
        --threads=$threads \
        --time=30 \
        run
done

```

- CPU performance is measured using the same prime-calculation workload in both environments.
- Ten repetitions provide data for calculating average performance and variation.
- Multiple thread counts allow the study of scalability as CPU demand increases.

9. Experiment 2 — Memory Performance

MEMORY EXPERIMENT	
Tool	→ Sysbench
Workload	→ Memory operations
Metrics	→ Operations/sec + latency
Tests	→ VM vs Container

VM:

```
mkdir -p ~/vm-vs-container-performance/results/raw/memory/vm

sysbench memory \
  --memory-block-size=1M \
  --memory-total-size=10G \
  --threads=4 \
  run
```

Save and repeat 10 times:

```
for i in {1..10}
do
  sysbench memory \
    --memory-block-size=1M \
    --memory-total-size=10G \
    --threads=4 \
    run > ~/vm-vs-container-performance/results/raw/memory/vm/run$i.txt
done
```

Docker:

```
mkdir -p ~/vm-vs-container-performance/results/raw/memory/container

docker run --rm \
  vm-container-benchmark \
  sysbench memory \
  --memory-block-size=1M \
  --memory-total-size=10G \
  --threads=4 \
  run
```

- This measures memory-operation performance using identical parameters.
- The workload helps identify differences in memory behavior and resource overhead.
- Repeat the test to obtain statistically useful measurements.

10. Monitor CPU and Memory Usage

In another terminal, run:

```
htop
```

Or:

```
vmstat 1
```

For Docker:

```
docker stats
```

- htop and vmstat provide system-level resource utilization information.
- docker stats reports CPU, memory, network I/O, and block I/O for running containers.
- Resource utilization should be recorded together with benchmark performance.

11. Experiment 3 — Disk I/O Performance

DISK EXPERIMENT	
Tool	→ fio
Operations	→ Read / Write
Workloads	→ Sequential / Random
Metrics	→ MB/s / IOPS / Latency

```
mkdir -p ~/fio-test
```

Sequential write:

```
fio --name=seq-write \
--filename=~/fio-test/testfile \
--size=2G \
--bs=1M \
--rw=write \
--direct=1 \
--iodepth=16 \
--runtime=30 \
--time_based
```

Sequential read:

```
fio --name=seq-read \
--filename=~/fio-test/testfile \
--size=2G \
--bs=1M \
--rw=read \
--direct=1 \
--iodepth=16 \
--runtime=30 \
--time_based
```

Random read:

```
fio --name=random-read \
--filename=~/fio-test/testfile \
--size=2G \
--bs=4k \
--rw=randread \
--direct=1 \
--iodepth=16 \
--runtime=30 \
--time_based
```

Random write:

```
fio --name=random-write \
--filename=~/fio-test/testfile \
--size=2G \
--bs=4k \
--rw=randwrite \
--direct=1 \
--iodepth=16 \
--runtime=30 \
--time_based
```

Save output:

```
mkdir -p results/raw/disk

fio --name=seq-write \
    --filename=~/.fio-test/testfile \
    --size=2G \
    --bs=1M \
    --rw=write \
    --direct=1 \
    --iodepth=16 \
    --runtime=30 \
    --time_based \
    > results/raw/disk/seq-write.txt
```

- fio measures storage throughput, IOPS, and latency.
- Sequential and random workloads are tested because storage behavior differs between access patterns.
- Use exactly the same fio parameters in the VM and container environments.

12. Run Disk Benchmark Inside Docker

```
mkdir -p ~/fio-test
```

```
docker run --rm \
  -v ~/fio-test:/fio-test \
  vm-container-benchmark \
  fio --name=seq-write \
  --filename=/fio-test/testfile \
  --size=2G \
  --bs=1M \
  --rw=write \
  --direct=1 \
  --iodepth=16 \
  --runtime=30 \
  --time_based
```

Random read:

```
docker run --rm \
  -v ~/fio-test:/fio-test \
  vm-container-benchmark \
  fio --name=random-read \
  --filename=/fio-test/testfile \
  --size=2G \
  --bs=4k \
  --rw=randread \
  --direct=1 \
  --iodepth=16 \
  --runtime=30 \
  --time_based
```

- The -v option mounts the host directory inside the container.
- This allows the container to access the benchmark storage path.
- Storage configuration must be equivalent between VM and container tests to avoid misleading results.

13. Experiment 4 — Network Performance

NETWORK EXPERIMENT	
Tool	→ iperf3
Metrics	→ Throughput / Retransmissions
Test	→ Client ↔ Server

On the server:

```
iperf3 -s
```

Find the server IP:

```
ip addr
```

On the client:

```
iperf3 -c <SERVER-IP> -t 30
```

Use multiple parallel streams:

```
iperf3 -c <SERVER-IP> -t 30 -P 4
```

Save the result:

```
iperf3 -c <SERVER-IP> -t 30 > results/raw/network/iperf3.txt
```

- One machine acts as the iperf3 server and the other as the client.
- Use the same client/server arrangement for VM and container tests.
- Document the network mode because NAT, bridged, and host networking can produce different results.

14. Experiment 5 — FastAPI Application

```
mkdir -p api  
cd api
```

Install FastAPI and Uvicorn:

```
pip install fastapi uvicorn
```

Create main.py:

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/health")  
def health():  
    return {"status": "healthy"}  
  
@app.get("/compute")  
def compute():  
    total = 0  
  
    for i in range(1_000_000):  
        total += i * i  
  
    return {"result": total}  
  
@app.get("/memory")  
def memory():  
    data = [i for i in range(1_000_000)]  
  
    return {  
        "elements": len(data)  
    }
```

Run:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

Test:

```
curl http://localhost:8000/health
```

Expected response:

STEP 14 - EXACT FASTAPI FILE LOCATION

When the original manual says 'Create main.py', first move into the api directory and then create the file there.

```
cd ~/vm-vs-container-performance
mkdir -p api
cd api
pwd
```

The final directory should be similar to:

```
/home/<your-ubuntu-user>/vm-vs-container-performance/api
```

Create main.py in this exact directory:

```
nano main.py
```

Paste the FastAPI code shown in the original manual. Save with Ctrl+O, press Enter, then exit with Ctrl+X.

Install the packages from the same api directory:

```
python3 -m pip install fastapi uvicorn
```

Start the API from the same api directory:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

Open a second Ubuntu Terminal for testing while the first terminal keeps Uvicorn running:

```
curl http://localhost:8000/health
```

- Do not close the terminal running Uvicorn while performing the API tests.
- If main.py is not found, run pwd and ls to confirm the file is inside the api directory.
- Keep the same application port and source code in the VM and Docker versions.

```
{"status": "healthy"}
```

- The API provides a realistic application workload in addition to synthetic benchmarks.
- The compute endpoint generates CPU activity and the memory endpoint generates memory activity.
- The same application code will be executed in the VM and in Docker.

15. Containerize the FastAPI Application

Create requirements.txt:

```
fastapi
uvicorn
```

Create the application Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build:

```
docker build -t performance-api .
```

Run:

```
docker run --rm \
  -p 8000:8000 \
  performance-api
```

Test:

```
curl http://127.0.0.1:8000/health
```

- The Dockerfile packages the FastAPI application and its dependencies into a reproducible image.
- Port 8000 connects the containerized API to the host.
- The VM and container now run the same application workload.

16. Benchmark the API

Install Apache Benchmark:

```
sudo apt install -y apache2-utils
```

Health endpoint:

```
ab -n 10000 -c 100 http://127.0.0.1:8000/health
```

STEP 15 - EXACT LOCATION FOR requirements.txt AND THE API DOCKERFILE

Keep the application files together inside the same api directory used for main.py.

```
cd ~/vm-vs-container-performance/api
pwd
nano requirements.txt
```

Enter the two required packages:

```
fastapi
uvicorn
```

Save with Ctrl+O, press Enter, and exit with Ctrl+X. Then create the API Dockerfile in the same api directory:

```
nano Dockerfile
```

Paste the API Dockerfile from the original manual and save it.

Build the application image from the project root:

```
cd ~/vm-vs-container-performance
docker build -t performance-api -f api/Dockerfile api
```

Run the API container with the controlled resource limits:

```
docker run --rm \
  --cpus=4 \
  --memory=8g \
  -p 8000:8000 \
  performance-api
```

Open another terminal and test it:

```
curl http://127.0.0.1:8000/health
```

- performance-api is the Docker image for the FastAPI workload; vm-container-benchmark is the general benchmark image.
- Keep the API Dockerfile inside api/ so the application remains separated from the benchmark environment.
- Use the same FastAPI source code in the VM and container versions.

Compute endpoint:

```
ab -n 1000 -c 10 http://127.0.0.1:8000/compute
```

Important metrics:

```
Requests per second
Time per request
Failed requests
Connection times
```

Optional: use wrk:

```
sudo apt install -y wrk

wrk -t4 -c100 -d30s http://127.0.0.1:8000/health
```

- Apache Benchmark and wrk generate concurrent requests against the application.
- Measure requests per second, latency, and failed requests.
- Use identical request counts and concurrency settings for VM and container comparisons.

17. Experiment 6 — Startup Time

Measure container startup:

```
time docker run --rm performance-api
```

A more controlled test:

```
time docker run --rm \
  -d \
  --name startup-test \
  -p 8000:8000 \
  performance-api
```

Stop it:

```
docker stop startup-test
```

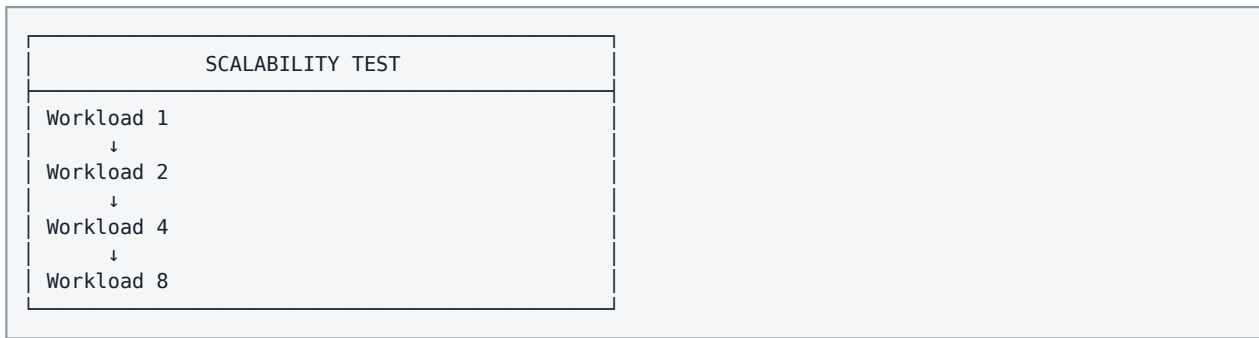
Record VM startup as:

```
VM startup:
Application ready:

Container startup:
Application ready:
```

- Startup time is important for cloud services, microservices, and rapid deployment.
- Perform multiple repetitions because startup time can vary.
- Compare both environment startup and application-ready time where possible.

18. Experiment 7 — Scalability



CPU scalability:

```
for threads in 1 2 4 8
do
    sysbench cpu \
        --cpu-max-prime=20000 \
        --threads=$threads \
        --time=30 \
        run
done
```

API scalability:

```
wrk -t1 -c10 -d30s http://127.0.0.1:8000/health
wrk -t2 -c50 -d30s http://127.0.0.1:8000/health
wrk -t4 -c100 -d30s http://127.0.0.1:8000/health
wrk -t4 -c200 -d30s http://127.0.0.1:8000/health
```

- Increasing workload shows how each environment behaves under increasing resource demand.
- Record throughput, latency, CPU usage, and memory usage at each workload level.
- The results can later be extended to Kubernetes replica and autoscaling experiments.

19. Automate the Benchmarks

Create a CPU benchmark script:

```
nano scripts/run_cpu.sh
```

Add:

```
#!/bin/bash

OUTPUT_DIR="../../results/raw/cpu"

mkdir -p "$OUTPUT_DIR"

for threads in 1 2 4 8
do
    echo "Running CPU test with $threads threads"

    sysbench cpu \
        --cpu-max-prime=20000 \
        --threads=$threads \
        --time=30 \
        run > "$OUTPUT_DIR/cpu_${threads}_threads.txt"

done

echo "CPU benchmark completed."
```

Make it executable:

```
chmod +x scripts/run_cpu.sh
```

Run:

```
./scripts/run_cpu.sh
```

- Automation ensures that benchmark parameters remain consistent.
- Scripts reduce manual errors and make repeated experiments easier.
- Create similar scripts for memory, disk, network, and application testing.

20. Collect Results in CSV

Create a processed CSV file such as:

```
results/processed/cpu_results.csv
```

Example CSV structure:

```
environment,threads,execution_time,events_per_second
VM,1,30.2,7450
VM,2,15.1,14920
Container,1,29.8,7520
Container,2,14.9,15040
```

- CSV files provide structured data that can be analyzed with Python, Excel, or other tools.
- Keep raw benchmark output separate from processed CSV files.
- Do not alter raw results to make them look cleaner.

21. Analyze Results Using Python

STEPS 19-20 - EXACT LOCATION FOR BENCHMARK SCRIPTS AND CSV RESULTS

Create benchmark scripts and processed result files from the project root.

```
cd ~/vm-vs-container-performance
mkdir -p scripts results/raw/cpu results/processed results/figures
pwd
```

Create the CPU automation script:

```
nano scripts/run_cpu.sh
```

Paste the original script, save with Ctrl+O, press Enter, and exit with Ctrl+X.

Make it executable and run it from the project root:

```
chmod +x scripts/run_cpu.sh
./scripts/run_cpu.sh
```

Create the processed CPU CSV from the project root:

```
nano results/processed/cpu_results.csv
```

Enter the measured values using the CSV structure shown in the original manual. Example values in the manual are only an example format and should not be copied as experimental results.

- Raw benchmark files remain in results/raw/ while cleaned values used by Python go into results/processed/.
- Use consistent column names for VM and container rows so the analysis scripts can group them correctly.
- Keep executable scripts inside scripts/ and do not mix scripts with benchmark result files.

Install analysis packages:

```
pip install pandas matplotlib numpy jupyter
```

Create the analysis script:

```
nano scripts/analyze_results.py
```

Example:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(
    "results/processed/cpu_results.csv"
)

print(df)

summary = df.groupby("environment")[
    "events_per_second"
].mean()

print("\nAverage CPU Performance:")
print(summary)

summary.plot(
    kind="bar",
    title="Average CPU Performance"
)

plt.ylabel("Events per Second")
plt.xlabel("Environment")
plt.tight_layout()

plt.savefig(
    "results/figures/cpu_performance.png",
    dpi=300
)

plt.show()
```

Run:

```
python3 scripts/analyze_results.py
```

- Pandas processes benchmark data and calculates summaries.
- Matplotlib creates graphs from the measured values.
- Saving figures to the repository makes the analysis visible on GitHub.

22. Calculate Statistical Values

STEPS 21-23 - EXACT LOCATION FOR PYTHON ANALYSIS FILES

Create the Python analysis scripts from the project root so their relative paths point to results/ correctly.

```
cd ~/vm-vs-container-performance
pwd
nano scripts/analyze_results.py
```

Paste the original analysis code, save with Ctrl+O, press Enter, and exit with Ctrl+X.

Run the analysis from the project root:

```
python3 scripts/analyze_results.py
```

Create a separate plotting script if required:

```
nano scripts/generate_plots.py
```

Save the plotting code there and run it from the project root:

```
python3 scripts/generate_plots.py
```

Open Jupyter from the project root when the notebook analysis is required:

```
cd ~/vm-vs-container-performance
jupyter notebook
```

- CSV inputs remain in results/processed/ and generated PNG graphs remain in results/figures/.
- Do not save analysis scripts inside results/; keep source code inside scripts/.
- Run scripts from the project root so relative paths continue to work.

```
import pandas as pd

df = pd.read_csv(
    "results/processed/cpu_results.csv"
)

summary = df.groupby("environment")[
    "events_per_second"
].agg([
    "mean",
    "median",
    "min",
    "max",
    "std"
])

print(summary)
```

- Mean represents average performance across repeated runs.
- Median helps reduce the influence of unusually high or low observations.
- Standard deviation indicates how much the measurements vary between runs.

23. Generate Comparison Graphs

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(
    "results/processed/cpu_results.csv"
)

for environment in df["environment"].unique():

    data = df[
        df["environment"] == environment
    ]

    plt.plot(
        data["threads"],
        data["events_per_second"],
        marker="o",
        label=environment
    )

plt.xlabel("Number of Threads")
plt.ylabel("Events per Second")
plt.title("CPU Performance vs Number of Threads")
plt.legend()
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "results/figures/cpu_scalability.png",
    dpi=300
)

plt.show()
```

Create equivalent graphs for:

CPU
Memory
Disk
Network
Application
Startup
Scalability

- Every graph should have a clear title, axis labels, units, and legend.
- Use actual measured values rather than assumed values.
- Graphs should support the numerical analysis rather than replace it.

24. Calculate Performance Difference

For execution time:

```
difference = (  
    (vm_time - container_time)  
    / vm_time  
) * 100  
  
print(  
    f"Performance difference: {difference:.2f}%"  
)
```

For throughput:

```
difference = (  
    (container_throughput - vm_throughput)  
    / vm_throughput  
) * 100  
  
print(  
    f"Throughput difference: {difference:.2f}%"  
)
```

- Use a formula appropriate to the metric being analyzed.
- Document the formula in the final report and apply it consistently.
- Do not call a percentage difference an overhead unless the calculation specifically represents overhead.

25. Final Comparison Table

Metric	VM	Container	Difference
CPU Performance	Actual	Actual	Calculated
Memory Usage	Actual	Actual	Calculated
Sequential Read	Actual	Actual	Calculated
Sequential Write	Actual	Actual	Calculated
Random Read	Actual	Actual	Calculated
Random Write	Actual	Actual	Calculated
Network Throughput	Actual	Actual	Calculated

Metric	VM	Container	Difference
Startup Time	Actual	Actual	Calculated
API Requests/sec	Actual	Actual	Calculated
API Latency	Actual	Actual	Calculated

- Populate this table only after collecting the actual measurements.
- Include units such as seconds, MB/s, IOPS, Mbps, requests/sec, and milliseconds.
- Use the table to summarize the experimental findings without hiding the raw measurements.

26. Final Project Structure

```

vm-vs-container-performance/
├── README.md
├── .gitignore
├── docs/
│   ├── architecture.png
│   ├── cpu-info.txt
│   ├── memory-info.txt
│   ├── storage-info.txt
│   └── methodology.md
├── vm/
│   ├── setup.sh
│   └── benchmark.sh
├── docker/
│   ├── Dockerfile
│   └── benchmark.sh
├── api/
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── workloads/
│   ├── cpu/
│   ├── memory/
│   ├── disk/
│   └── network/
├── scripts/
│   ├── run_cpu.sh
│   ├── run_memory.sh
│   ├── run_disk.sh
│   ├── run_network.sh
│   ├── collect_metrics.py
│   ├── analyze_results.py
│   └── generate_plots.py
├── results/
│   ├── raw/
│   ├── processed/
│   └── figures/
└── analysis/
    └── analysis.ipynb

```

- The structure separates code, application files, raw results, processed data, graphs, and documentation.

- A clean structure makes the repository easier to understand and reproduce.
- The same organization can later be extended for Kubernetes experiments.

27. Create the README

```
# Performance Analysis of Virtual Machines and Containers

## Abstract

## Objectives

## Research Questions

## Experimental Environment

## Hardware Configuration

## Software Configuration

## Architecture

## Methodology

## CPU Experiment

## Memory Experiment

## Disk I/O Experiment

## Network Experiment

## Application Experiment

## Startup-Time Experiment

## Scalability Experiment

## Results

## Statistical Analysis

## VM vs Container Comparison

## Discussion

## Limitations

## Conclusion

## Future Work

## Reproduction Instructions
```

- The README is the main documentation page of the project.
- Include the architecture diagram, methodology, commands, and important graphs.
- A new user should be able to understand and reproduce the experiment from the README.

28. GitHub — Create the Repository

Create a new repository on GitHub:

```
GitHub
↓
New Repository
↓
Repository Name
↓
Create Repository
```

Suggested repository name:

```
vm-vs-container-performance
```

- Keep the repository name short and descriptive.
- Add a short description explaining that the project compares VM and container performance.
- The repository can contain source code, documentation, benchmark data, and graphs.

29. Initialize Git Locally

Open a terminal in the project root:

```
cd ~/vm-vs-container-performance
```

Initialize Git:

```
git init
```

Check:

```
git status
```

- `git init` creates a local Git repository inside the project folder.
- `git status` shows which files are new, modified, or untracked.
- Always inspect the status before committing.

30. Create .gitignore

Create the file:

```
nano .gitignore
```

Add:

```
__pycache__/  
*.pyc  
  
.venv/  
venv/  
  
.env  
  
.ipynb_checkpoints/  
  
*.log  
  
.vscode/  
.idea/  
  
*.vmdk  
*.vmem  
*.vmss  
*.nvram
```

- The .gitignore file prevents unnecessary or sensitive files from being uploaded.
- Do not upload passwords, API keys, credentials, or private configuration files.
- Large VMware virtual disk files should not normally be pushed to GitHub.

31. Add the Project Files

```
git add .
```

Check what is staged:

```
git status
```

Create the first commit:

```
git commit -m "Initial project setup"
```

- git add stages project files for the next commit.
- git status lets you verify that unwanted files are not staged.
- The commit creates the first version of the project history.

32. Create the GitHub Repository

```
GitHub  
↓  
New Repository  
↓  
Repository Name  
↓  
Create Repository
```

After creating the repository, GitHub will provide the repository URL. Use that exact URL in the next command.

- Do not manually construct the repository URL if GitHub has already provided it.
- The local repository will be connected to this remote repository.
- Keep the repository public if you want to showcase the project in your portfolio.

STEPS 27-32 - EXACT GITHUB FILE AND TERMINAL LOCATION

Run all Git commands from the project root. Do not initialize separate Git repositories inside api/, docker/, scripts/ or results/.

```
cd ~/vm-vs-container-performance  
pwd
```

Create README.md at the project root:

```
nano README.md
```

Create .gitignore in the same project root:

```
nano .gitignore
```

Paste the ignore entries from the original manual and save them.

Initialize the repository and make the first commit:

```
git init  
git status  
git add .  
git status  
git commit -m "Initial project setup"  
git branch -M main
```

- README.md and .gitignore belong directly inside ~/vm-vs-container-performance.
- The entire project should be one Git repository.
- Use git status before committing so unwanted files can be removed or ignored.

33. Connect the Local Project to GitHub

```
git remote add origin <GITHUB-REPOSITORY-URL>
```

Verify:

```
git remote -v
```

Set the main branch:

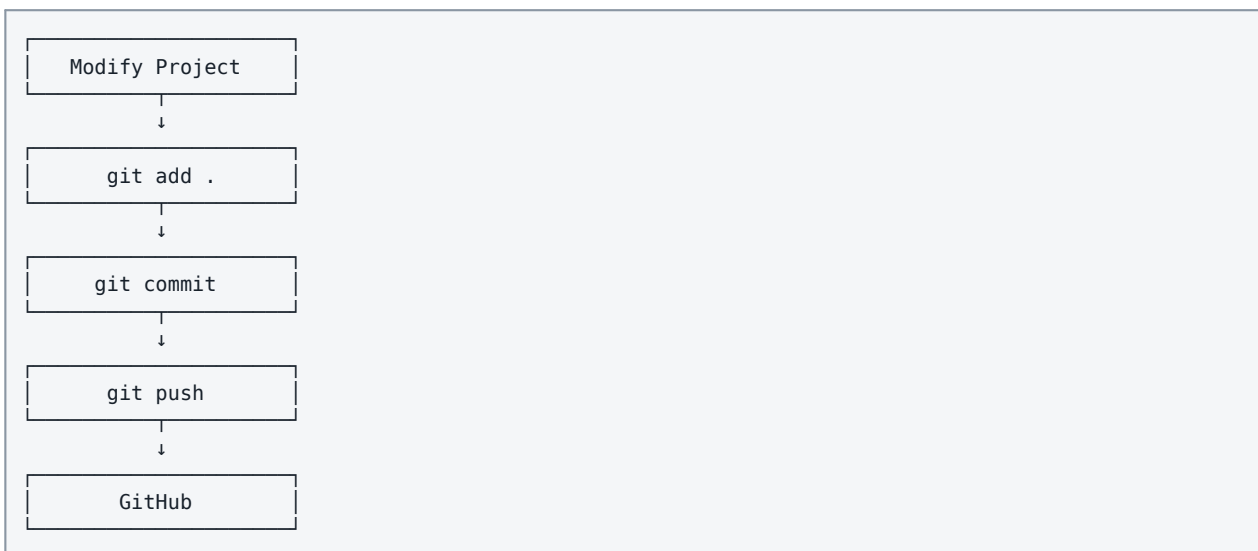
```
git branch -M main
```

- origin identifies the GitHub repository connected to the local project.
- main is used as the primary branch.
- Use the exact URL generated by GitHub.

34. Push the Project to GitHub

```
git push -u origin main
```

Complete Git workflow:



- git push uploads the committed project to GitHub.
- After the first push, future updates normally use git add, git commit, and git push.
- Refresh the repository page to verify that the project files and documentation are visible.

35. Future Updates to GitHub

Whenever a new experiment or analysis is completed:

```
git status
git add .
git commit -m "Add CPU benchmark results"
git push
```

Example meaningful commit messages:

STEPS 33-35 - EXACT GITHUB PUSH LOCATION

Continue from the same Ubuntu terminal with the project root as the current directory.

```
cd ~/vm-vs-container-performance
pwd
git remote add origin <GITHUB-REPOSITORY-URL>
git remote -v
git branch -M main
git push -u origin main
```

For later experiment updates, remain in the same directory:

```
git status
git add .
git commit -m "Add CPU benchmark results"
git push
```

Examples of meaningful later commits:

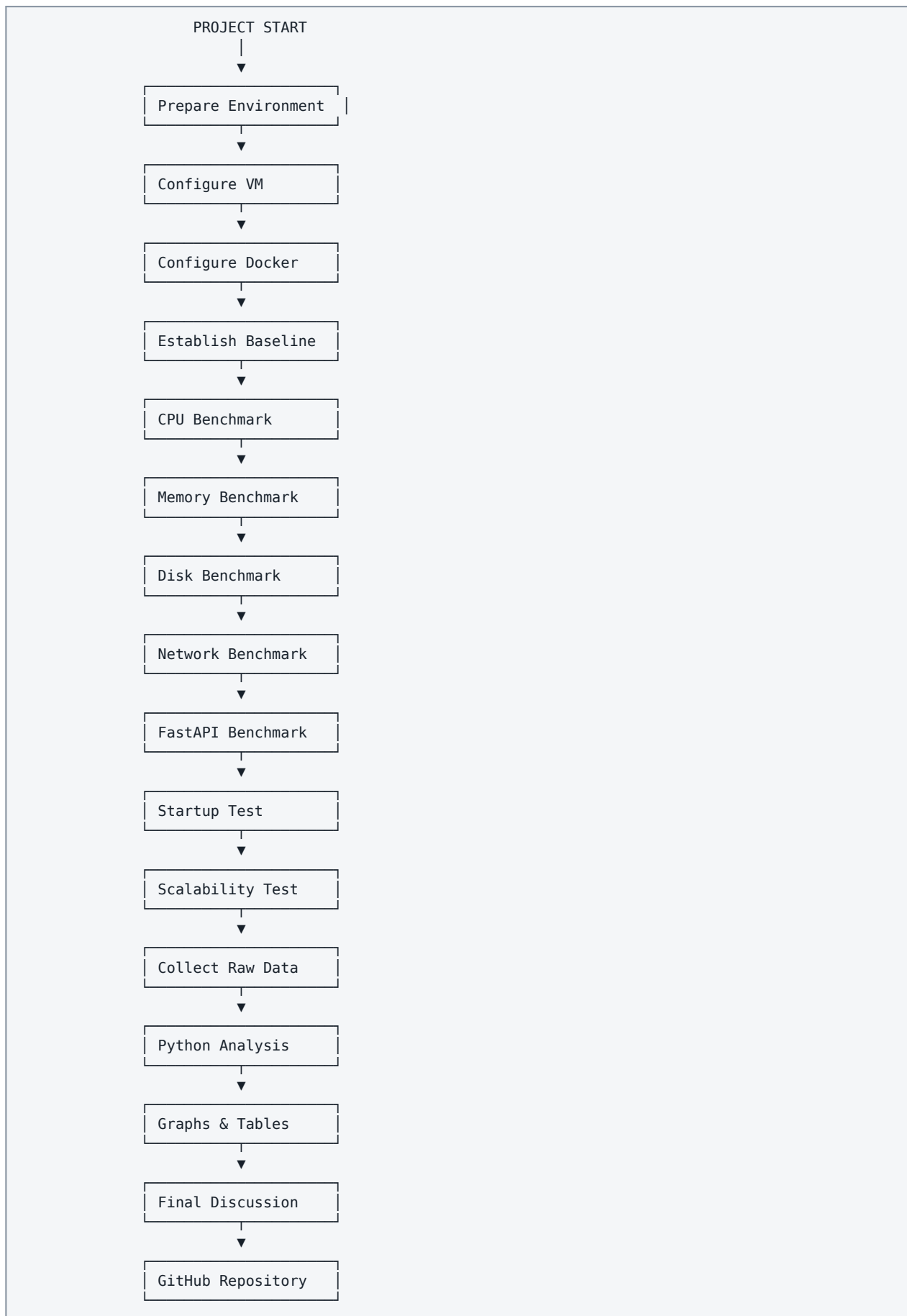
```
git commit -m "Add memory benchmark"
git commit -m "Add disk I/O benchmark"
git commit -m "Add network benchmark"
git commit -m "Add FastAPI workload"
git commit -m "Add performance analysis"
git commit -m "Add benchmark graphs"
git commit -m "Update README"
```

- The GitHub URL is used only in the git remote add origin command.
- Only committed changes are uploaded by git push.
- Never commit credentials, API keys, VM disk files or other large temporary files.

```
git commit -m "Add CPU benchmark"
git commit -m "Add memory benchmark"
git commit -m "Add disk I/O benchmark"
git commit -m "Add network benchmark"
git commit -m "Add FastAPI workload"
git commit -m "Add performance analysis"
git commit -m "Add benchmark graphs"
git commit -m "Update README"
```

- Meaningful commits create a clear development history.
- Commit after completing important parts of the experiment instead of uploading everything only at the end.
- GitHub then acts as both the project repository and a record of how the experiment was developed.

36. Complete Experimental Workflow



- Complete the project sequentially instead of running all experiments at once.
- Verify each result before moving to the next experiment.

- Keep raw measurements, scripts, graphs, and documentation together so the final GitHub repository is reproducible.

Important Experimental Practice

Do not assume beforehand that VMs or containers will always perform better. The purpose of this project is to measure the actual behavior under controlled conditions. Your final conclusion should be based on the collected measurements, repeated runs, statistical analysis, and documented configuration.

STUDENT FILE-LOCATION MAP

Use these exact locations throughout the experiment

Project root: ~/vm-vs-container-performance

VM documentation: docs/

General benchmark Dockerfile: docker/Dockerfile

FastAPI source: api/main.py

FastAPI packages: api/requirements.txt

FastAPI Dockerfile: api/Dockerfile

Benchmark automation: scripts/

Raw benchmark results: results/raw/

Processed CSV data: results/processed/

Graphs: results/figures/

Jupyter notebook: analysis/analysis.ipynb

Git files: README.md and .gitignore at the project root

Before using any relative path, verify the current directory:

```
cd ~/vm-vs-container-performance
pwd
```

If the output does not end in vm-vs-container-performance, return to the project root before continuing.

- Use this page together with the original lab manual steps.
- The original manual pages remain unchanged in the revised PDF; these additional pages clarify the exact file and terminal locations.
- Actual hardware settings, benchmark outputs and measured performance values must come from the student's own experiment.